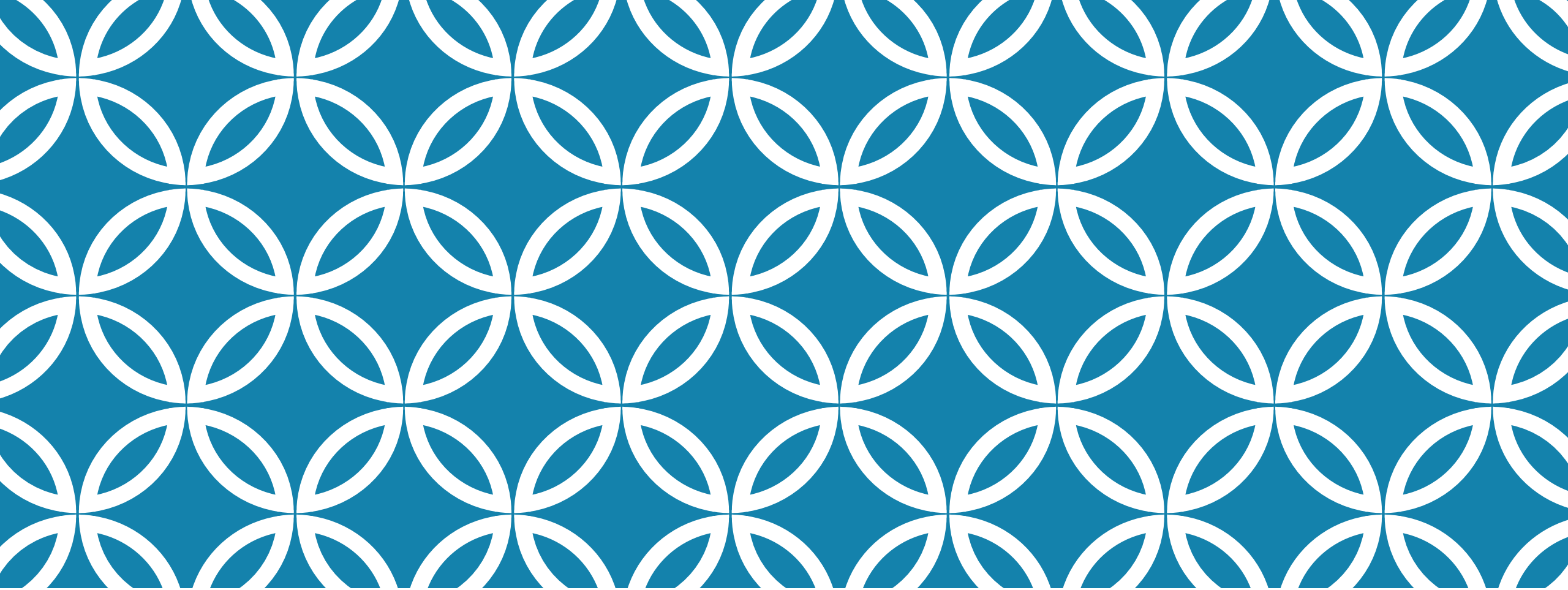


COMPLEXIDADE DE PROBLEMAS

COMPLEXIDADE EM TEMPO

Assumindo que o problema possa ser solucionado por algoritmos:

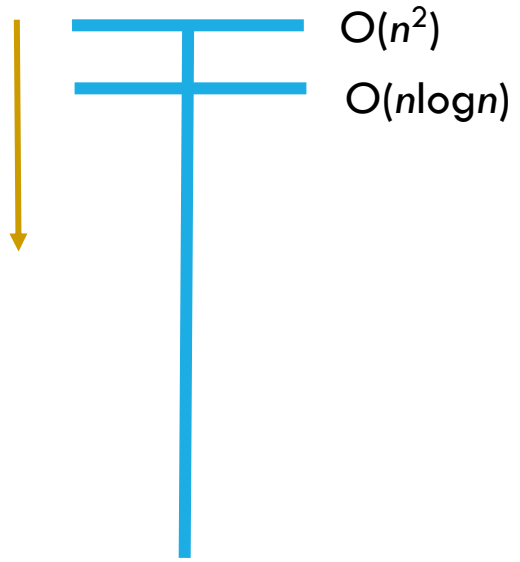
O ***LIMITE SUPERIOR*** da complexidade em tempo de um problema refere-se ao ***tempo no pior caso do melhor algoritmo*** para resolver este problema.

Exemplo: Ordenação por comparação

Mergesort – $O(n\log n)$

Quicksort – $O(n^2)$

O limite superior da complexidade da ordenação é $O(n\log n)$.

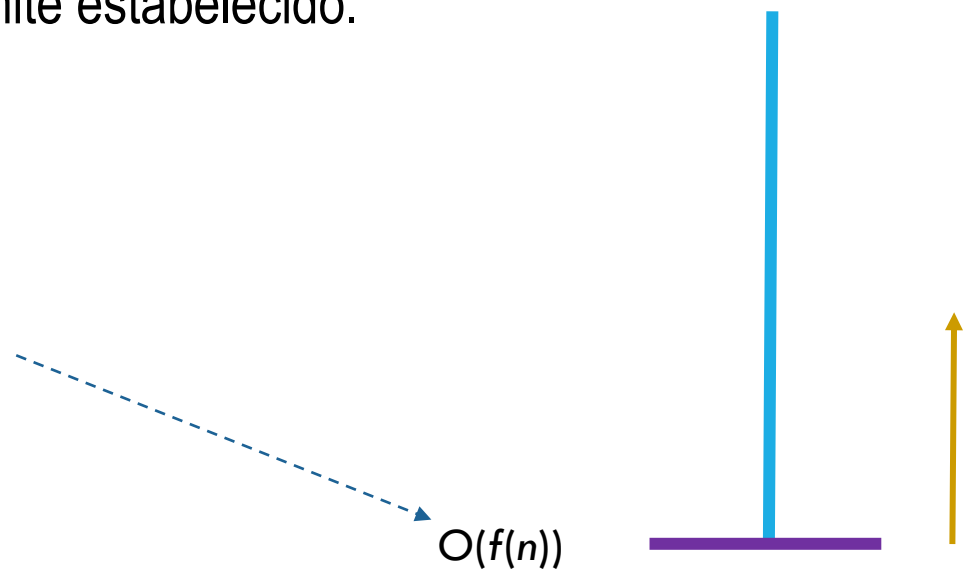


COMPLEXIDADE EM TEMPO

O ***limite inferior*** da complexidade em tempo de um problema refere-se à melhor complexidade possível.

É um resultado teórico que determina não ser possível desenvolver um algoritmo ***cuja complexidade de pior caso*** seja menor que um certo limite estabelecido.

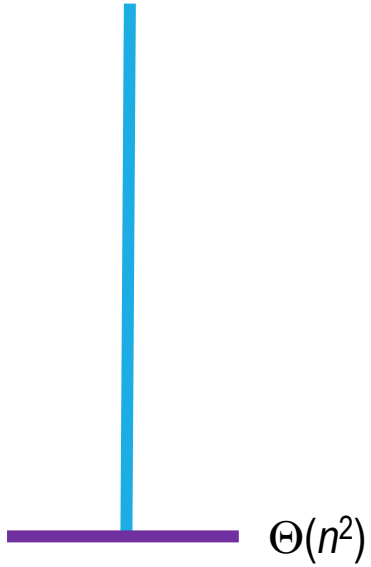
Um algoritmo para o problema terá pior caso, pelo menos



COMPLEXIDADE EM TEMPO

Exemplo: Soma e multiplicação de matrizes quadradas de ordem n.

O algoritmo precisa acessar $\Theta(n^2)$ elementos das matrizes. Portanto, um ***limite inferior*** é $\Theta(n^2)$.



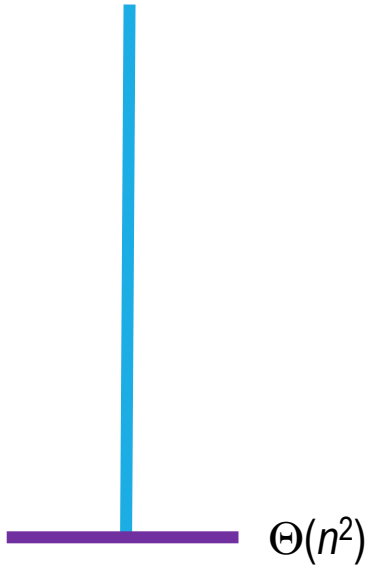
COMPLEXIDADE EM TEMPO

Exemplo: Algoritmo para soma de matrizes quadradas de ordem *n*.

```
Soma(A,B,C,n)
    para i ← 1 até n
        para j ← 1 até n
            C[i,j] ← A[i,j] + B[i,j]
```

Complexidade: $\Theta(n^2)$

Limite superior = Limite inferior

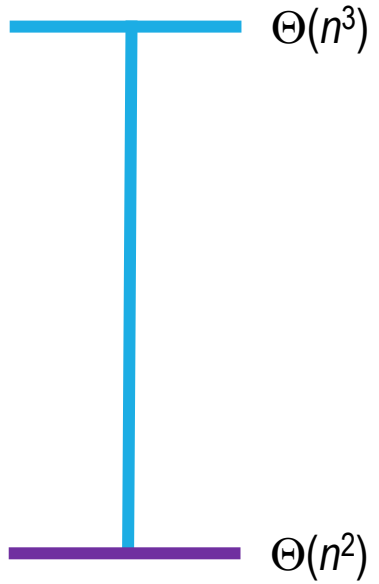


COMPLEXIDADE EM TEMPO

Exemplo: Algoritmo para multiplicação de matrizes quadradas de ordem n.

```
Soma(A,B,C,n)
    para i ← 1 até n
        para j ← 1 até n
            C[i,j] ← 0
            para k ← 1 até n
                C[i,j] ← C[i,j] + A[i,k] x B[k,j]
```

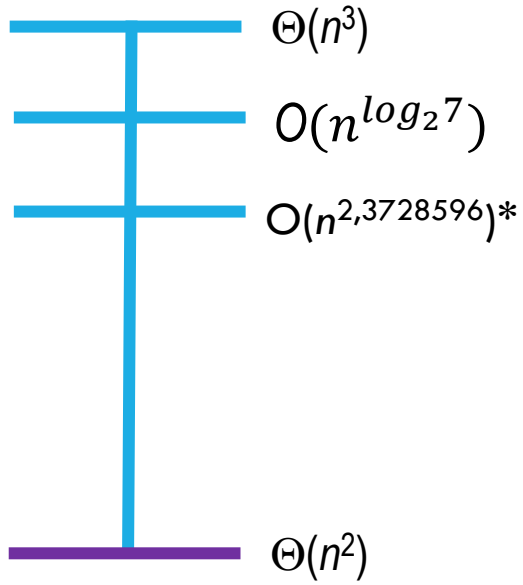
Complexidade: $\Theta(n^3)$



COMPLEXIDADE EM TEMPO

Exemplo: Algoritmo para multiplicação de matrizes quadradas de ordem *n*.

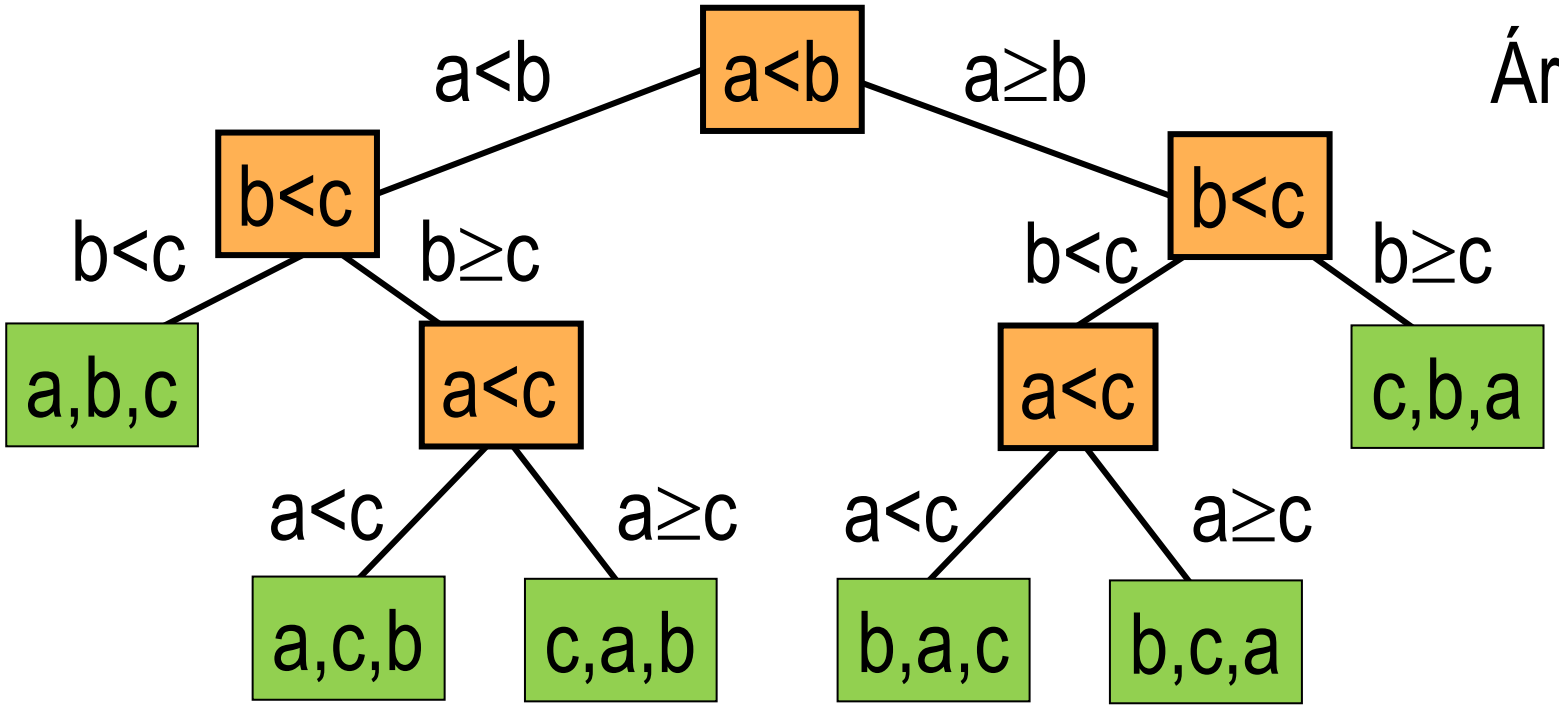
Algoritmo de Straβen: $O(n^{log_2 7}) \approx O(n^{2,807})$



** Alman, J., Williams, V. V. (2020), "A Refined Laser Method and Faster Matrix Multiplication", 32nd Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2021), arXiv:2010.05846*

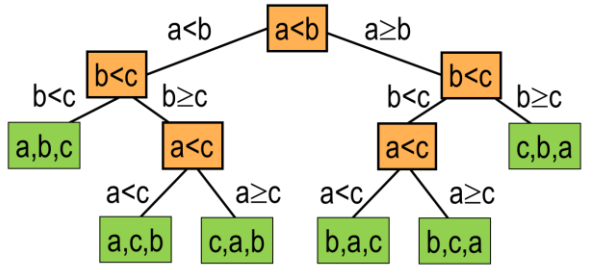
COMPLEXIDADE EM TEMPO

Exemplo: Ordenação por comparação - Ordenar três números inteiros a, b e c.



Árvore de Decisão

COMPLEXIDADE EM TEMPO



Exemplo: Ordenação por comparação - Ordenar três números inteiros a, b e c.

A complexidade é a profundidade de uma folha na árvore de decisão.

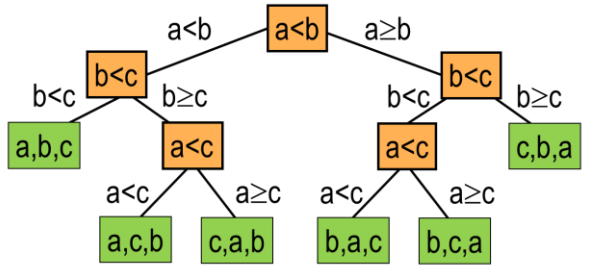
A **complexidade de pior caso** é a **maior profundidade** de uma folha **=** altura da árvore.

Relação entre o número de folhas e a altura de uma árvore binária completa é **#folhas = 2^{altura}**

#folhas = *n*!, onde *n* é o número de elementos a serem ordenados.

Como a árvore de decisão não é necessariamente completa temos que: $n! \leq 2^{\text{altura}}$

COMPLEXIDADE EM TEMPO



Exemplo: Ordenação por comparação - Ordenar três números inteiros a, b e c.

$n! \leq 2^{\text{altura}}$

$\log_2 n! \leq \log_2 2^{\text{altura}}$

$\text{altura} \geq \lceil \log_2 n! \rceil$

Aproximação de Stirling: $\lceil \log_2 n! \rceil = n \log_2 n - \frac{n}{\ln 2} + \frac{1}{2} \log_2 n + O(1)$

A complexidade de pior caso é dada pela altura da árvore. Assim: $O(n \log_2 n)$
Portanto, o limite inferior para o problema é $O(n \log n)$.

COMPLEXIDADE EM TEMPO

Existem limites inferiores naturais, como, por exemplo, o tamanho da entrada uma vez que o algoritmo deverá ler sua entrada.

Exemplo: limite inferior para soma de matrizes e para multiplicação de matrizes quadradas de ordem n é $\Omega(n^2)$.

Um limite inferior para um problema P, a função I, é tal que a complexidade de pior caso de qualquer algoritmo que resolva P é $\Omega(I)$.

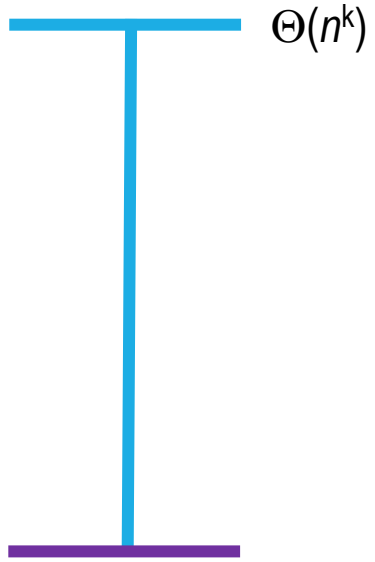
Se existir algoritmo A tal que A é $O(I)$, então A é dito ***algoritmo ótimo para P***.

PROBLEMA TRATÁVEL

Um **problema** é dito ***tratável*** se existe um algoritmo polinomial que o resolva, ou seja, possui limite superior polinomial.

Exemplo de problemas tratáveis:

- Ordenação
- Busca em lista linear
- Inserção em árvore binária de busca
- Árvore geradora mínima de um grafo



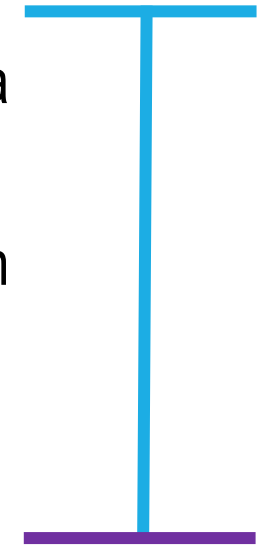
PROBLEMA INTRATÁVEL

Um **problema** é dito ***intratável*** se, comprovadamente, não existe um algoritmo polinomial que o resolva, ou seja, possui limite inferior exponencial.

Causas da intratabilidade:

1. O problema é tão difícil que é requerido tempo exponencial para encontrar uma solução.

2. A solução é tão extensa que não pode ser descrita com uma expressão com comprimento limitado por uma função polinomial no tamanho da entrada.

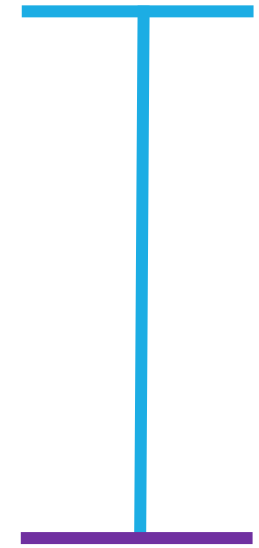


PROBLEMA INTRATÁVEL

Exemplo de problemas intratáveis:

Causa 2. Encontrar todos os *tours* com comprimento menor ou igual a *B* em um problema do Caixeiro Viajante.

Obs. Neste curso, vamos nos restringir a primeira causa de intratabilidade.

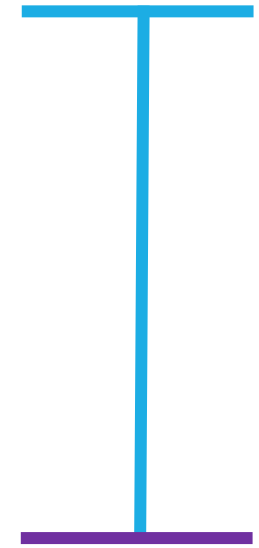


PROBLEMA INTRATÁVEL

Causa 1.

Problemas indecidíveis

Problemas decidíveis em uma máquina de Turing não determinística.



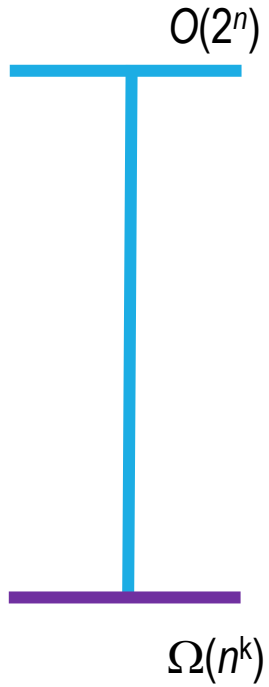
COMPLEXIDADE DE PROBLEMAS

Alguns problemas não se encaixam em nenhuma das duas classes anteriormente definidas pois:

- São conhecidos apenas algoritmos exponenciais que os resolvem, ou seja, ***possuem limite superior exponencial***.

- ***Possuem limite inferior polinomial***

Não existe uma prova de que o melhor algoritmo que possa ser desenvolvido para o problema tenha complexidade exponencial.

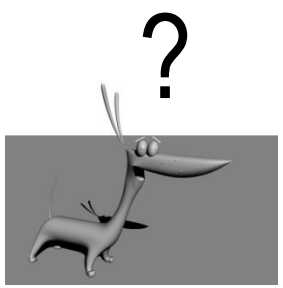


PROBLEMAS DE DECISÃO

Decidir sobre a existência de uma configuração S que atenda as restrições de Π.

Resposta: Sim ou não







PROBLEMAS DE DECISÃO

Exemplos

1. A lista L está ordenada?
2. Existe caminho menor que *k* no grafo *G*?
3. *G* é um grafo hamiltoniano?



PROBLEMAS DE LOCALIZAÇÃO E OTIMIZAÇÃO

Localização

Localizar ou exibir uma configuração S que atenda a resposta sim do problema de decisão.

Otimização

Exibir, se existir, uma configuração S que satisfaça as propriedades exigidas em Π e otimize uma função z(x)

CLASSE P



Problema de decisão para o qual existe um ***algoritmo determinístico polinomial em tempo*** que o resolva.

$O(n^k)$

1. O problema de decidir se um grafo *G* é Euleriano pertence à classe P?

2. O problema de decidir se um grafo *G* pode ser colorido com 2 cores pertence à classe P?

3. O problema de decidir se um grafo *G* é Hamiltoniano pertence à classe P?

CLASSE P



Para provar que um problema de decisão Π pertence à classe P, basta apresentar um algoritmo que resolva Π em tempo polinomial.

1. O problema de decidir se um grafo *G* é Euleriano pertence à classe P.

2. O problema de decidir se um grafo *G* pode ser colorido com 2 cores pertence à classe P.

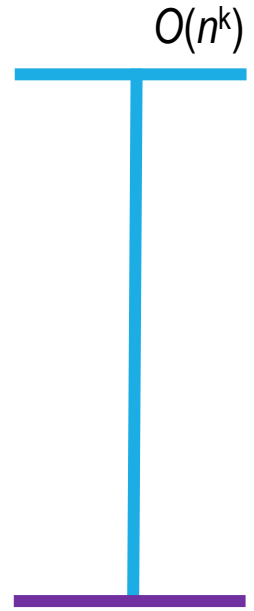
3. O problema de decidir se um grafo *G* é Hamiltoniano não pertence à classe P.

$O(n^k)$

CLASSE P

A classe P é fechada em relação ao complemento.

A classe C de problemas é *fechada em relação ao complemento* se para todo problema $\Pi \in C$, o complemento de Π (Π_{comp}) também pertence à C.



Problema Π: Decidir se um grafo é 2-colorível. (Pertence à classe P)

Problema Π_{comp}: Decidir se um grafo é não-2-colorível. (Pertence à classe P)

Algoritmo de busca em profundidade decide se o grafo é ou não 2-colorível.

ALGORITMO DE VERIFICAÇÃO

Um algoritmo de dois argumentos *x* e *y*.

x é uma instância do problema de decisão

y é uma solução de *x*

O algoritmo verifica se *y* é válida para *x*.

VERIFICAÇÃO EM TEMPO POLINOMIAL

Suponha uma dada instância da versão de decisão do problema do caminho de tamanho *k* em grafos:

- Existe um caminho *P* de comprimento menor ou igual a *k* entre os vértices *s* e *t* no grafo *G*?

VERIFICAÇÃO EM TEMPO POLINOMIAL

Suponha uma dada instância da versão de decisão do problema do caminho de tamanho *k* em grafos:

- Existe um caminho *P* de comprimento menor ou igual a *k* entre os vértices *s* e *t* no grafo *G*?

Dado que alguém apresentou o caminho *P* no grafo *G* com comprimento menor ou igual a *k*, você é capaz de verificar se a resposta “SIM” é verdadeira em tempo polinomial?

É necessário apresentar um algoritmo que faz a verificação da resposta sim, mostrando que ele é correto e polinomial.

VERIFICAÇÃO EM TEMPO POLINOMIAL

Exercício. Fazer o algoritmo de verificação do problema de caminho de tamanho *k*.



VERIFICAÇÃO EM TEMPO POLINOMIAL

- Existe um ciclo Hamiltoniano de comprimento *c* no grafo *G*?

Dado que alguém apresentou o ciclo no grafo *G* com comprimento *c*, você é capaz de verificar se a resposta “SIM” é verdadeira em tempo polinomial ?

Exercício. Fazer o algoritmo de verificação para o problema de ciclo Hamiltoniano.

Mostre que o algoritmo é correto e polinomial.



CLASSE NP



Problema de decisão para o qual existe um algoritmo de verificação que verifica a resposta "**SIM**" em tempo polinomial.

Exemplos: Problemas de decisão:

-Existe caminho entre *s* e *t* no grafo *G* com comprimento menor ou igual a *k*?

-Existe ciclo hamiltoniano no grafo *G*?

-A lista L está ordenada?

-As retas A e B se interceptam no ponto x?

OBS: $P \subseteq NP$

ALGORITMO NÃO-DETERMINÍSTICO POLINOMIAL

Consiste de duas fases

1. Adivinhação



2. Verificação



ALGORITMO NÃO-DETERMINÍSTICO POLINOMIAL

Fase de adivinhação

Cadeia de caracteres y é gerada.

y corresponde (ou não) a uma solução para o problema de decisão.

y está no formato da solução do problema.

y foi gerada em tempo polinomial.



ALGORITMO NÃO-DETERMINÍSTICO POLINOMIAL

Fase de verificação

Algoritmo determinístico verifica:

- se a solução y gerada está no formato próprio para o problema.

-verifica se y é solução para o problema. (Resposta SIM)



O algoritmo deve fazer a verificação em tempo polinomial.

CLASSE NP

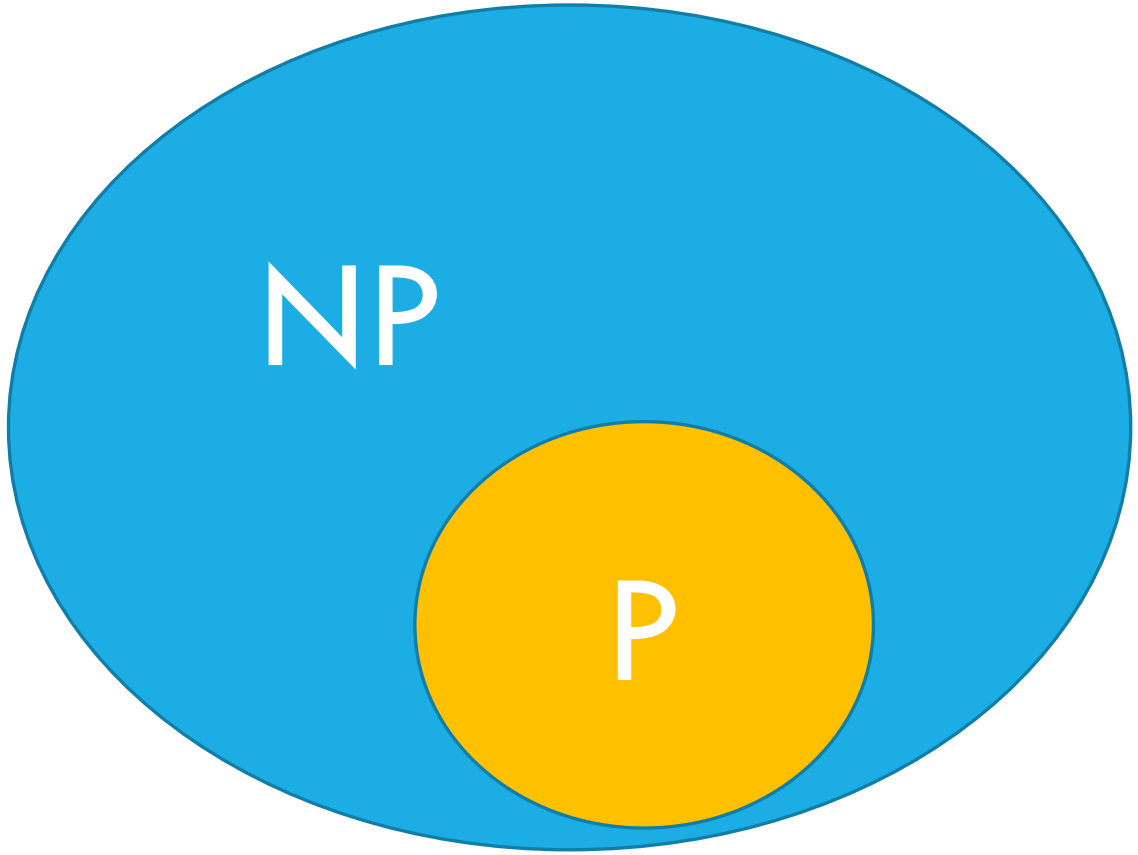
Compreende os problemas de decisão para os quais existe um algoritmo não-determinístico polinomial em relação ao tamanho de sua entrada.

CLASSES P E NP - RESUMO

P é a classe de problemas de decisão que podemos *decidir* ou *resolver* utilizando um algoritmo determinístico que roda em tempo polinomial.

NP é a classe de problemas de decisão para os quais checamos ou verificamos sua solução (resposta SIM) através de um algoritmo determinístico que roda em tempo polinomial.

CLASSES P E NP - RESUMO



PROBLEMAS NP-COMPLETOS

Problemas mais difíceis da classe NP

Se um deles for resolvido por um algoritmo determinístico polinomial, então

$$P = NP$$

Para ter uma melhor noção de quando um problema é mais difícil que outro, vamos ver antes o conceito de ***redução de problemas***.

REDUÇÃO

Suponha que temos um procedimento que transforma qualquer instância *a* do problema *A* em uma instância *b* do problema *B* com as seguintes características:

1. A transformação é feita em tempo polinomial
2. As respostas são as mesmas. Isto é, uma resposta é “sim” para *a* se e somente se a resposta para *b* também é “sim”.

Chamamos tal procedimento de ***algoritmo de redução em tempo polinomial***.



REDUÇÃO



Problemas de decisão: Π e Π'

Π ***se reduz polinomialmente*** a Π' (notação: $\Pi \ \alpha_p \ \Pi$'), se existe um algoritmo determinístico polinomial *Alg* que transforma a instância *I* de Π na instância *I'* de Π' de modo que a resposta para *I* é SIM se e somente se a resposta para *I'* é SIM.

Uma redução de um problema Π em um problema Π' é um algoritmo polinomial *f* que transforma toda instância *I* de Π em uma instância *f*(*I*) de Π' e um algoritmo polinomial *h* que mapeia toda solução *s* de *f*(*I*) em uma solução *h*(*s*) de *I*.

A existência destes dois procedimentos, *f* e *h*, implica que qualquer algoritmo que resolva Π' pode ser convertido em um algoritmo para Π, com os procedimentos *f* e *h*.

REDUÇÃO

Vamos considerar um problema de decisão, A, que gostaríamos de resolver em tempo polinomial.

Vamos considerar um problema de decisão, B, que sabemos resolver em tempo polinomial.

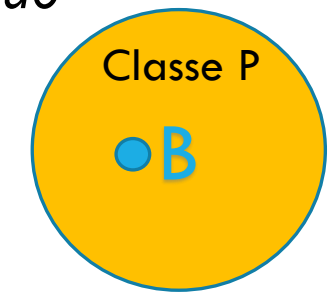
Considere que existe um algoritmo polinomial, Alg, que reduz A a B.

REDUÇÃO

Vamos considerar um problema de decisão, *A*, que gostaríamos de resolver em tempo polinomial.

Verificando se um problema desconhecido A pertence à classe P, partindo do princípio que o problema conhecido B pertence à classe P.

$A \in P?$

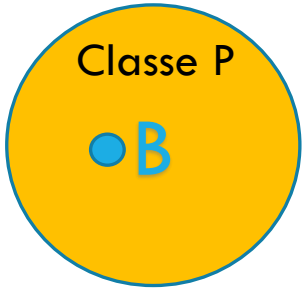


Para resolver *A*, fazemos:

1. Dada uma instância *a* de *A*, use o algoritmo de redução em tempo polinomial para transformá-la em *b* de *B*.
2. Resolva *b* em tempo polinomial.
3. Use a resposta de *b* como resposta de *a*.

REDUÇÃO

A ∈ P?



Como cada etapa demora tempo polinomial, as três juntas demoram tempo polinomial, e temos um algoritmo polinomial para *A*.

Portanto, $A \in P$.



PROBLEMA NP-COMPLETO

Um problema de decisão Π é dito ***NP-completo*** se

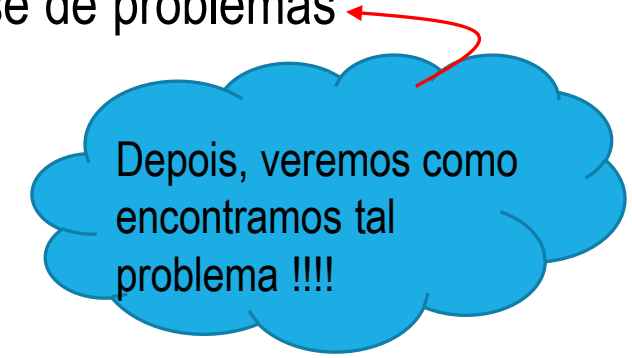
1. $\Pi \in$ NP

2. todo problema Π' em NP, $\Pi' \alpha_p \Pi$

PROBLEMA NP-COMPLETO

A relação α_p é transitiva.

Suponha que já conhecemos um problema de decisão Π que pertence à classe de problemas NP-completos.



Queremos provar que Π' também é NP-completo.

Suponha que temos um algoritmo *Alg* que faz a redução em tempo polinomial de Π em Π'.

PROBLEMA NP-COMPLETO

Agora podemos construir uma prova para mostrar que Π', também, é NP-completo.

Como Π é um caso particular de Π' e Π está em NP-completo, então, Π' também está na mesma classe.

SAT: O PRIMEIRO PROBLEMA NP-COMPLETO

SAT: “Satisfiability”

Dada uma fórmula booleana na forma normal conjuntiva, encontrar uma atribuição de valores para as variáveis tal que o resultado seja “verdadeiro” ou mostrar que tal atribuição não existe.

Exemplo:

$$(x \vee y \vee z) \wedge (x \vee \neg y) \wedge (y \vee \neg z) \wedge (z \vee \neg x) \wedge (\neg x \vee \neg y \vee \neg z)$$

SAT: O PRIMEIRO PROBLEMA NP-COMPLETO

TEOREMA DE COOK

SAT é NP-completo

Todos os problemas da classe NP podem ser reduzidos polinomialmente à SAT.

Conclusão: Se SAT está em P, NP = P.

Obs. Prova – livro Cormen et al.

PROBLEMA NP-DIFÍCIL

Um problema Π é dito ***NP-difícil*** se

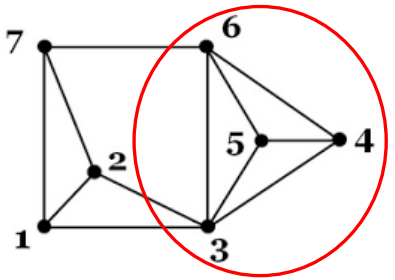
todo problema Π' em NP, $\Pi' \alpha_p \Pi$

EXEMPLO: REDUÇÃO A PROBLEMA NP-COMPLETO

Dado que o problema 3-CNF é NP-completo, provar que o problema de decidir se um grafo tem um clique de tamanho *k* é NP-completo.

Problema 3-CNF: Uma fórmula booleana está em forma normal conjuntiva (**c**onjunctive **n**ormal **f**ormula) se é expressa por um grupo de cláusulas com o conector AND e cada uma dessas cláusulas é formada por literais com conectores OR. *Uma fórmula é 3-CNF se cada uma das cláusulas tem exatamente três literais distintos*.

Problema da *k*-clique: Decidir se um grafo *G* possui um subgrafo completo de ordem *k*.



Clique de ordem $k=4$

EXEMPLO: REDUÇÃO A PROBLEMA NP-COMPLETO

1. Para provar que Clique é NP-completo, primeiro deve-se mostrar que Clique está em NP.

A verificação se V', $|V'| = k$, é uma clique de tamanho k em $G = (V,E)$ pode ser feita em tempo polinomial verificando se para todo $u, v \in V'$ a aresta $(u,v) \in E$.

Portanto, Clique pertence a NP.

2. Provar que 3-CNF se reduz polinomialmente à Clique (Clique é NP-difícil).

CLIQUE É NP-DIFÍCIL

O algoritmo de redução começa com uma instância de de 3-CNF:

$$\phi = C_1 \wedge C_2 \wedge \ldots \wedge C_k \, .$$

Para $r = 1, \ldots, k$, cada cláusula C_r tem 3 literais distintos, l_{1r}, l_{2r} e l_{3r}.

Constrói-se um grafo G tal que:

ϕ é satisfeita se e somente se G tem uma clique de tamanho k.

CLIQUE É NP-DIFÍCIL

Construção do grafo $G = (V,E)$

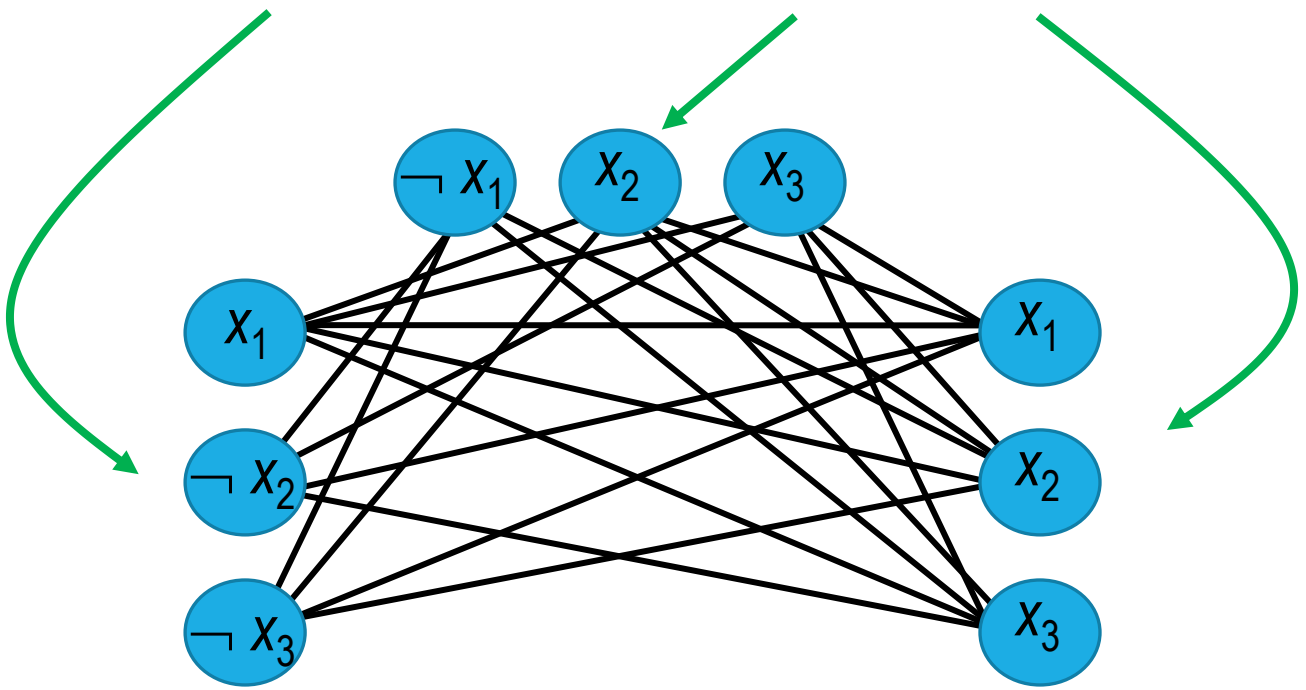
Para cada cláusula $C_r = (l'_{1r} \vee l'_{2r} \vee l'_{3r})$ são criados 3 vértices v_{1r} , v_{2r} e v_{3r} em V. Uma aresta é inserida entre dois vértices v_{ir} e v_{js} se ambas as características a seguir se mantem:

1. v_{ir} e v_{js} estão em triplas diferentes, isto é $r \neq s$.
2. Seus literais correspondentes são coerentes, isto é, l'_{ir} não é a negação de l'_{js} .

CLIQUE É NP-DIFÍCIL

Exemplo:

$$\phi = (x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3)$$



CLIQUE É NP-DIFÍCIL

Devemos mostrar que esta transformação é uma redução.

$\Rightarrow$

Suponha que ϕ tenha uma atribuição satisfatória.

Então cada cláusula C_r contem um literal l'_{ir} que recebe a atribuição de 1 e cada um desses literais corresponde a um vértice v_{ir}.

Escolhendo-se um literal “verdadeiro” de cada cláusula, formaremos um conjunto V' de k vértices, sendo uma clique.

CLIQUE É NP-DIFÍCIL

$\Leftarrow$

Por outro lado suponha que *G* tem uma clique *V'* de *k* vértices.

Nenhuma aresta em *G* conecta vértices da mesma tripla, assim *V'* contem exatamente um vértice de cada tripla.

Pode-se atribuir 1 a cada literal correspondente, uma vez que um literal e seu complemento jamais receberão 1 simultaneamente.

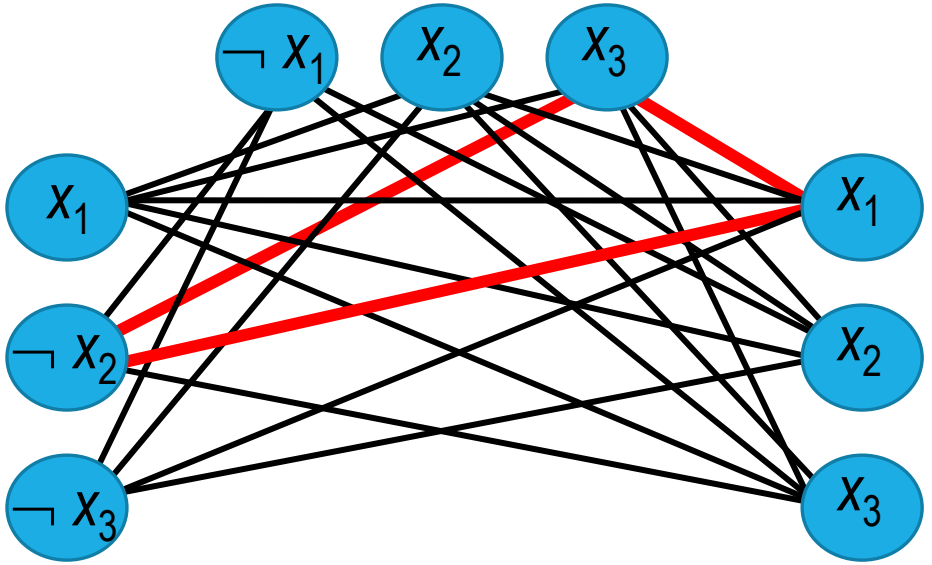
Cada cláusula é satisfeita, assim ϕ é satisfeita.

Qualquer variável que não pertença a uma clique pode receber qualquer valor.

CLIQUE É NP-DIFÍCIL

Exemplo:

$$\phi = (x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3)$$

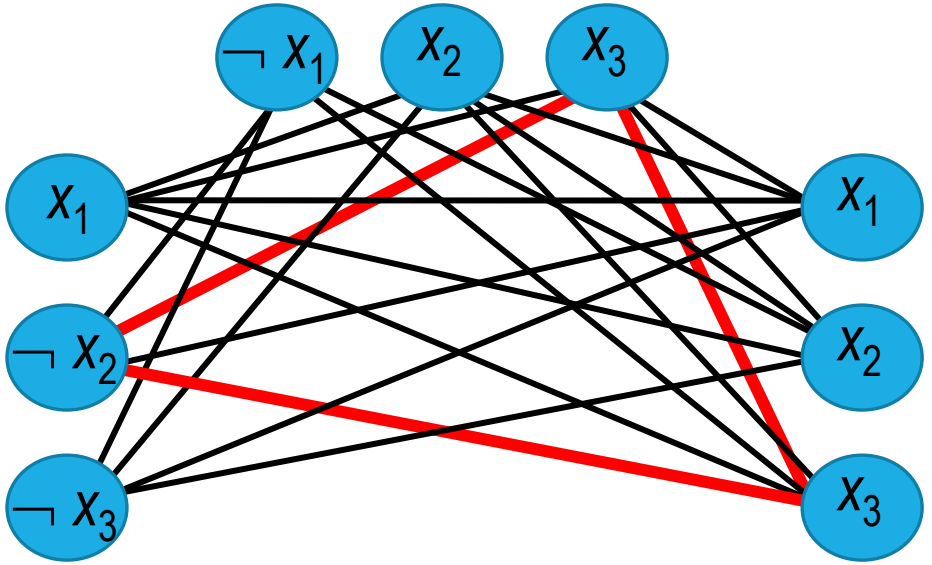


$x_1 = 1; x_2 = 0; x_3 = 1$ satisfaz ϕ

CLIQUE É NP-DIFÍCIL

Exemplo:

$$\phi = (x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3)$$



x_1 = 0 ou 1; x_2 = 0; x_3 = 1 satisfaz ϕ

CLIQUE É NP-COMPLETO

Uma vez que mostramos que para pelo menos uma classe de grafos o problema de Clique é NP-difícil, então podemos afirmar que Clique é NP-difícil. Como Clique também pertence a NP, Clique é NP-completo.

PROBLEMAS DE OTIMIZAÇÃO



Problemas de Otimização não podem ser NP-Completos!

NP-Completos são definidos no contexto dos problemas de DECISÃO.



EXERCÍCIO: PROBLEMA NP-COMPLETO

A partir da informação de que o Problema do Somatório é NP-completo, mostre que o Problema da Partição também é NP-completo.

Problema do Somatório de Subconjuntos:

"Dado um conjunto de inteiros, existe um subconjunto não vazio cuja soma dos elementos é igual a um valor *v* especificado?" Por exemplo, para o conjunto A = {-7, -3, -2, 5, 8} e para *v* = 0, a resposta seria "SIM", uma vez que o subconjunto A' = {-3, -2, 5} apresenta soma dos elementos igual a 0.

Problema da Partição:

"Dado um conjunto de inteiros S, S pode ser particionado nos conjuntos X e X' = S – X, de maneira que o somatório dos elementos em X seja igual ao somatório dos elementos em X'?"

EXERCÍCIO: PROBLEMA NP-COMPLETO

Para mostrar que um problema Π é NP-completo, temos 5 passos a percorrer:

1) Mostrar que Π é um problema de decisão;

2) Apresentar um algoritmo polinomial que, dada uma solução para a resposta “SIM” de Π, verifique se essa resposta é válida;

3) Mostrar a redução de um problema NP-completo Π’ a Π;

4) A redução de Π’ a Π deve ocorrer em tempo polinomial;

5) O problema original Π deve ter uma solução se, e somente se, Π’ tem uma solução.

EXERCÍCIO: PROBLEMA NP-COMPLETO

Para mostrar que um problema Π é NP-completo, temos 5 passos a percorrer:

1) Mostrar que Π é um problema de decisão;

2) Apresentar um algoritmo polinomial que, dada uma solução para a resposta “SIM” de Π, verifique se essa resposta é válida;

3) Mostrar a redução de um problema NP-completo Π’ a Π;

4) A redução de Π’ a Π deve ocorrer em tempo polinomial;

5) O problema original Π deve ter uma solução se, e somente se, Π’ tem uma solução.

EXERCÍCIO: PROBLEMA NP-COMPLETO

1) Partição é um problema de decisão

O problema da Partição é de decisão, uma vez que admite como respostas possíveis apenas:

- “SIM” (caso seja possível particionar S da maneira especificada)
- “NÃO” (caso não seja possível particionar S da maneira especificada).



EXERCÍCIO: PROBLEMA NP-COMPLETO

2) Algoritmo polinomial que, dada uma solução para a resposta “SIM” de Π, verifica se essa resposta é válida

Dada uma partição do conjunto S em dois conjuntos X e X’ que satisfaz a resposta “SIM” do Problema da Partição, é possível verificar, em tempo polinomial se esta resposta satisfaz o problema, por meio do seguinte algoritmo, de complexidade O($n$log$n$), a depender do método de ordenação empregado:

EXERCÍCIO: PROBLEMA NP-COMPLETO

```
VerificaSolucaoParticao (S, tamS, X, tamX, X', tamX')
  se (tamX + tamX' ≠ tamS) então retorne falso;
  somaX ← somaX' ← 0; ctrX ← ctrX' ← 0;
  Ordena (S); Ordena (X); Ordena (X');
para i ← 0 até tamS faça
    achou ← falso;
    se (ctrX < tamX) então
      se (S[i] = X[ctrX]) então
          somaX ← somaX + S[i]; ctrX++; achou ← verdadeiro;
    se (ctrX' < tamX' && !achou) então
      se (S[i] = X'[ctrX']) então
         somaX' ← somaX' + S[i]; ctrX'++; achou ← verdadeiro;
   se (!achou) então retorne falso;
   se (somaX = somaX') então retorne verdadeiro;
   senão retorne falso;
```



EXERCÍCIO: PROBLEMA NP-COMPLETO

3) Qualquer problema NP-completo Π' deve ser reduzido a Π;

Se um problema NP-completo Π' se reduz polinomialmente ao problema Π, então todo problema em NP se reduz polinomialmente a Π.

Dado que o problema do Somatório de Conjuntos é NP-completo, basta, então, mostrar a ***redução polinomial do Somatório de Conjuntos ao problema da Partição***, como a seguir.

EXERCÍCIO: PROBLEMA NP-COMPLETO

3) Qualquer problema NP-completo Π' deve poder ser reduzido a Π;

Seja *X* um conjunto de inteiros e *t* um valor especificado.

O problema do Somatório de Conjuntos se preocupa com a existência de um subconjunto *Y* de *X* tal que a soma dos elementos em *Y* seja exatamente *t*.

Seja *s* a soma dos elementos em *X*. Defina $X' = X \cup \{s - 2t\}$. Solucione o Problema da Partição tendo *X'* como entrada. A resposta do Problema do Somatório de Conjuntos será a mesma do Problema de Partição solucionado.



EXERCÍCIO: PROBLEMA NP-COMPLETO

4) A redução de Π' para Π deve ocorrer em tempo polinomial;

A redução mencionada em (3) ocorre em tempo polinomial, pois consiste apenas na adição de um elemento calculado em tempo polinomial ao conjunto de entrada do Problema da Partição.



EXERCÍCIO: PROBLEMA NP-COMPLETO

5) O problema original Π deve ter uma solução se, e somente se, Π' tem uma solução.

Utilizando a redução especificada em (3), existe uma solução para o problema original (Somatório de Conjuntos) se, e somente se, existe uma solução para o Problema da Partição, ou seja,

$\langle X, t \rangle \in$ ao Problema do Somatório de Conjuntos
se e somente se
$\langle X' \rangle \in$ ao Problema da Partição.

Note que a soma dos elementos em X' é $s' = s + (s - 2t) = 2s - 2t$.

EXERCÍCIO: PROBLEMA NP-COMPLETO

$\rightarrow$: se $\langle X, t \rangle \in$ ao Problema do Somatório de Conjuntos
então
$\langle X' \rangle \in$ ao Problema da Partição.

$\rightarrow$: Se existe um conjunto de números *C1* em *X* tal que a soma seja *t*, então os elementos restantes *C2* em *X* somam *s* – *t*.

Portanto, existe uma partição de *X'* em dois conjuntos, *C3* e *C4*,

$$X' = C3 \cup C4,\ C3 \cap C4 = \varnothing$$

cada um com soma *s* – *t*, definidos como a seguir: $C3 = C1 \cup \{s - 2t\}$; $C4 = C2$.

EXERCÍCIO: PROBLEMA NP-COMPLETO

$\leftarrow$: se $\langle X' \rangle \in$ ao Problema da Partição
então
$\langle X, t \rangle \in$ ao Problema do Somatório de Conjuntos.

$\leftarrow$: Digamos que exista uma partição de X' em dois conjuntos, $C3$ e $C4$, cada um com soma dos elementos igual a $s - t$. Uma dessas partições, por exemplo, $C3$, contém o número $s - 2t$. A remoção do número $s - 2t$ do conjunto $C3$ faz com que a soma dos elementos em $C3$ passe a ser $(s - t) - (s - 2t) = t$.

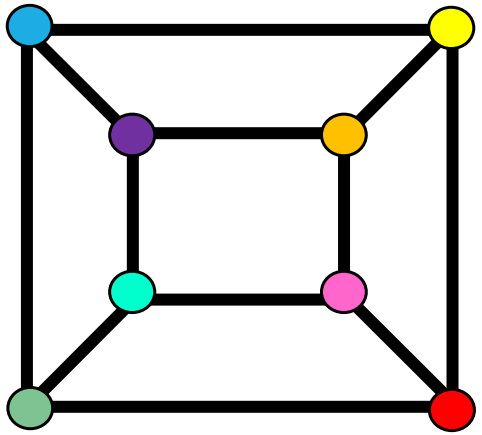
Além disso, todos os elementos em $C3$ após tal remoção fazem parte de X, entrada para o problema do Somatório de Conjuntos.

Portanto, $C5 = C3 \setminus \{s - 2t\}$ é uma solução para o Problema do Somatório de Conjuntos tendo como entrada o conjunto X.

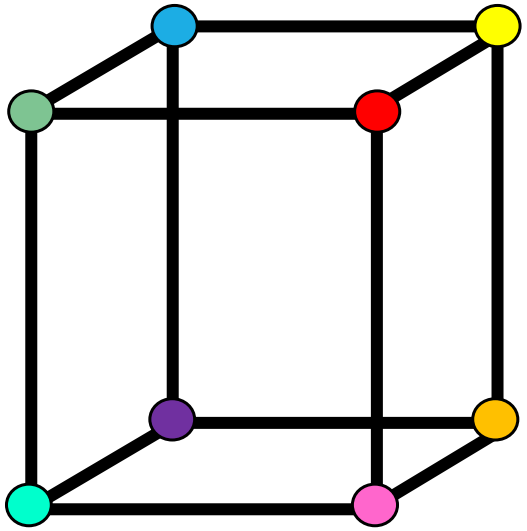


ISOMORFISMO DE GRAFOS

Dois grafos $G_1=(N_1,M_1)$ e $G_2=(N_2,M_2)$ são *isomorfos* se existe correspondência entre os seus vértices e arestas de tal maneira que existe uma função unívoca $f : N_1 \rightarrow N_2$ tal que (i,j) é elemento de M_1 se e somente se $(f(i),f(j))$ é elemento de M_2.

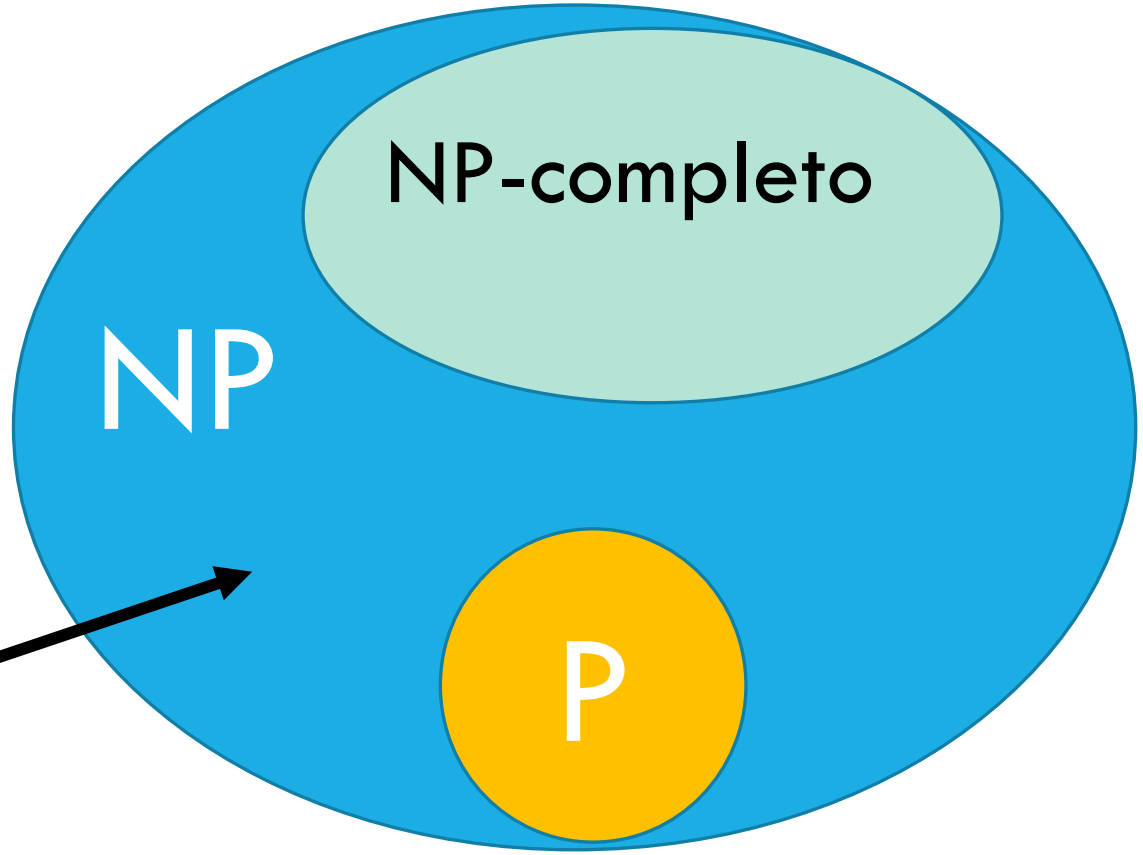


G_1



G_2

ISOMORFISMO DE GRAFOS



Isomorfismo

Obs. Classe NPI: NP indeterminados. Não se apresentou algoritmo polinomial, e também não se provou ser NP-completo.

CLASSE CO-NP



Problema de decisão para o qual existe um algoritmo de verificação que verifica a resposta "**NÃO**" em tempo polinomial.

OBS:

NP – Verifica resposta sim em tempo polinomial (certificado de pertinência)

Co-NP – Verifica resposta não em tempo polinomial (certificado de desqualificação)

OBS: P $\subseteq$ Co-NP

CLASSE CO-NP



Exemplo: O grafo *G* é não-hamiltoniano?

Este problema está em co-NP.

Dada a resposta não, e o ciclo hamiltoniano em *G*, a verificação pode ser feita em tempo polinomial.

CLASSE CO-NP



Exemplo: Dado inteiro positivo *a*, *a* é primo?

Este problema está em co-NP.

Dada a resposta não, e os fatores de *a*, a verificação é imediata.



Este problema está em NP?

CLASSE CO-NP



Exemplo: Dados inteiro positivo *a*, *a* é primo?

Este problema está em NP.

Resultado da Teoria dos Números

Teorema. Um inteiro $p > 2$ é primo se e somente se existe um inteiro r, $1 < r < p$, tal que $r^{p-1} = 1 \bmod p$ e para todo q divisor primo de $p - 1$, $r^{(p-1)/q} \neq 1 \bmod p$.

Prova: Pratt, V. R. (1975). Every prime has a succint certificate. SIAM Journal of Computing 4(3), 214-220.

CLASSE CO-NP



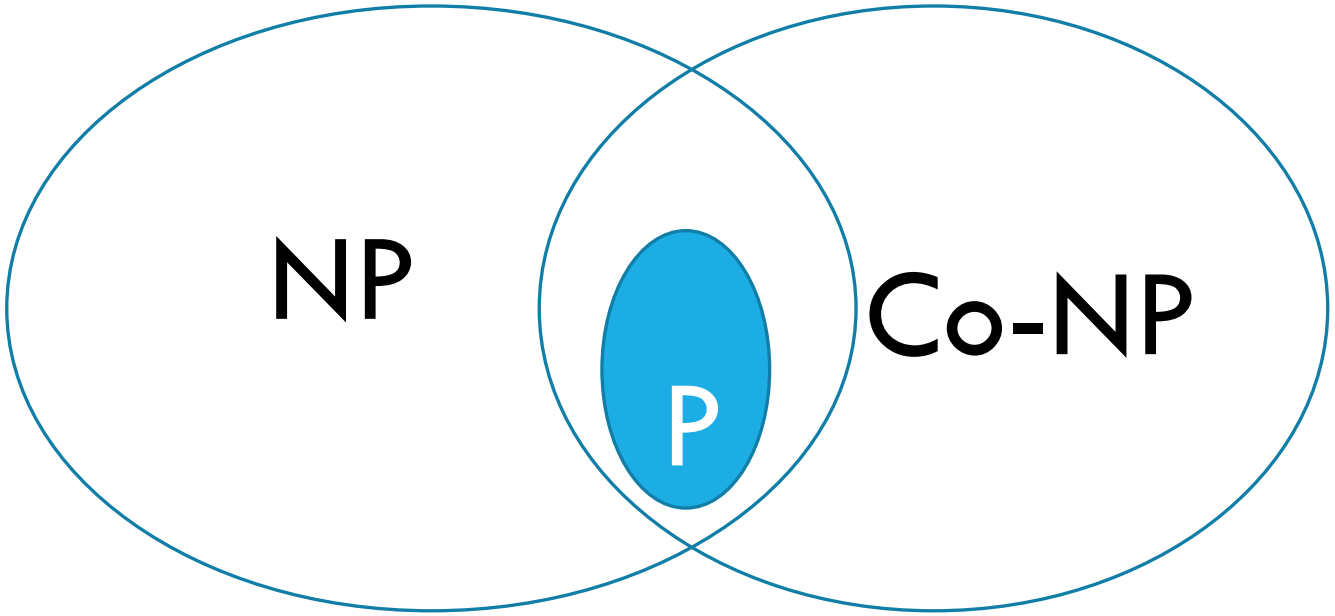
Exemplo: Dados inteiro positivo *a*, *a* é primo?

Este problema está em P.

Prova. *Agrawal, M., Kayal, N., Saxena, N.* (2004). *"PRIMES is in P". Annals of Mathematics.* **160** (2): *781–793.*

NP ∩ CO-NP

Se um problema está em NP∩Co-NP, aceita-se, em geral, que existe uma forte evidência de que este problema não é NP-completo (caso contrário NP=Co-NP).



NP ∩ CO-NP

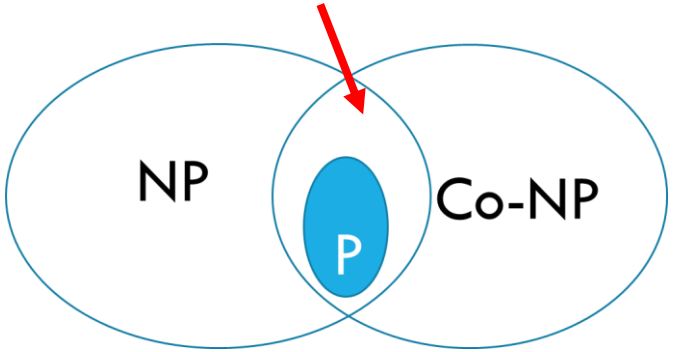
Exemplo: Fatoração de Inteiros

Dados inteiros positivos *a* e *b*, $1 < b < a$, a possui fator *d* tal que $1 < d \leq b$?

Este problema está em NP: Dada a resposta sim e *d*, a verificação é imediata.

Este problema está em co-NP: Dada a resposta não e a fatoração de *a* em primos distintos maiores que *b,* a verificação é imediata.

Não foi exibido algoritmo polinomial que resolva o problema.



PROBLEMA CO-NP-COMPLETO

Um problema de decisão Π é dito **co-*NP-completo*** se

1. $\Pi \in$ co-NP

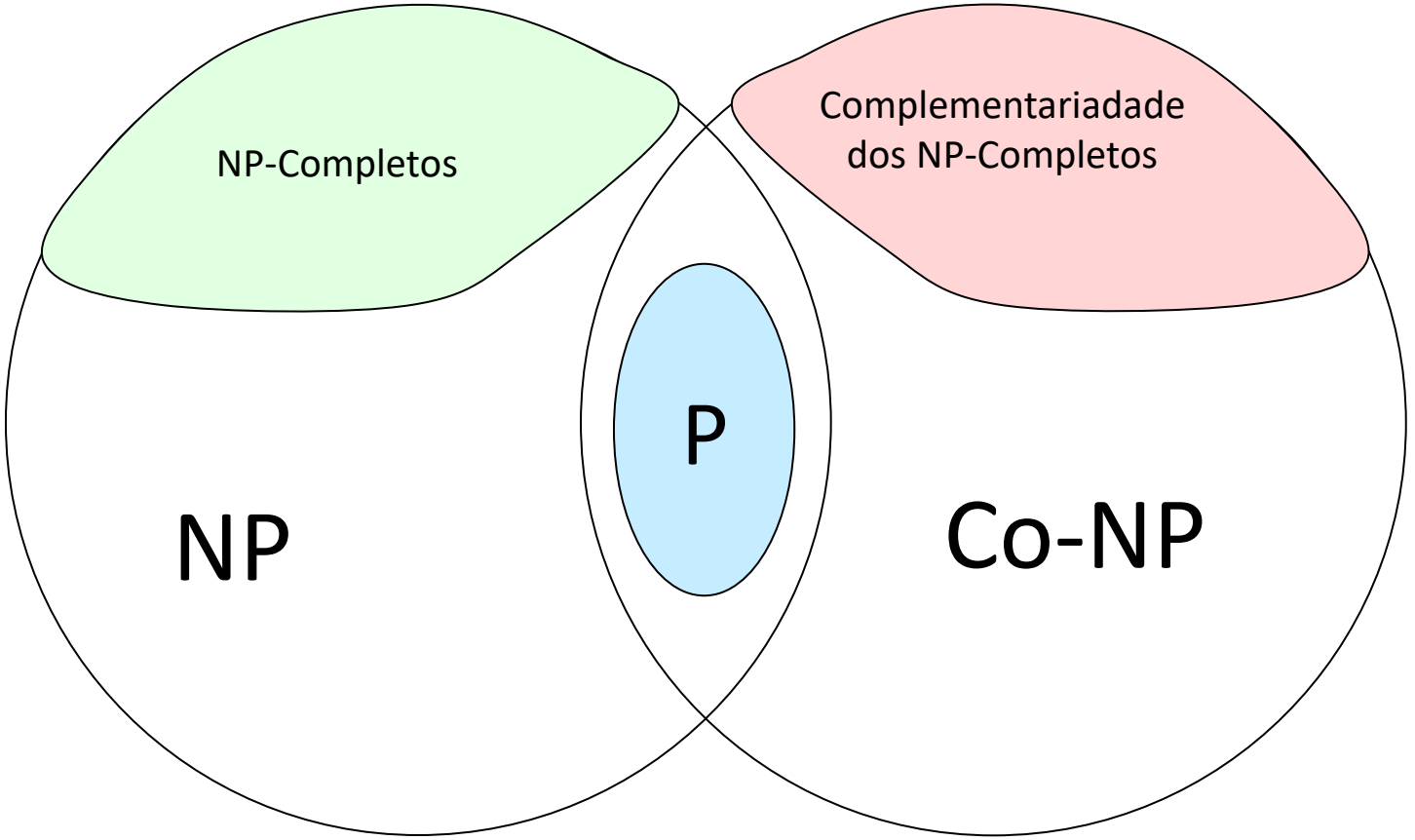
2. todo problema Π' em co-NP, $\Pi' \alpha_p \Pi$

PROBLEMA CO-NP-COMPLETO

Exemplo: Problema da Tautologia

Determinar se uma fórmula Booleana é uma tautologia, isto é, se qualquer atribuição de valores aos literais leva a uma avaliação “Verdadeira” da fórmula.

CLASSES NP E CO-NP



SIMETRIA NP X CO-NP

1. Não existe simetria

2. O problema de afirmar que um circuito booleano não satisfaz a exigência de constituição lógica *não pode ser verificado em passo polinomial*.
   Dizer que sim PODE!

ALÉM DE NP



Exptime

Problemas de decisão que podem ser resolvidos em tempo exponencial em uma máquina de Turing determinística.

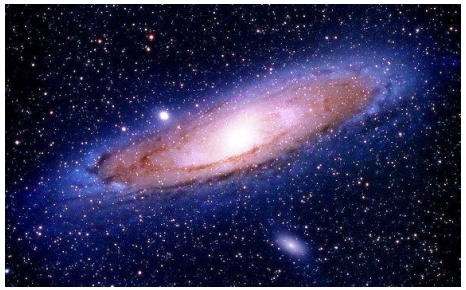
Nexptime

Problemas de decisão que podem ser resolvidos em tempo exponencial em uma máquina de Turing não-determinística.

$$P \subseteq NP \subseteq Exptime \subseteq Nexptime$$

$$P \subset Nexptime$$

ALÉM DE NP



L (NL)

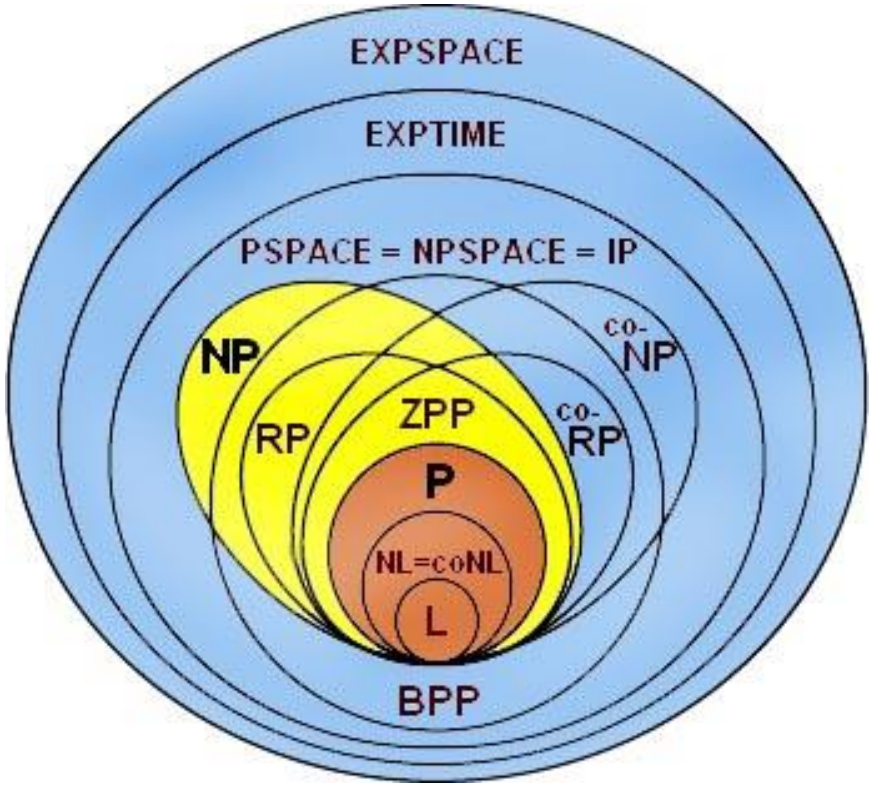
Problemas de decisão que podem ser resolvidos em espaço logarítmico em uma máquina de Turing determinística (não-determinística).

PSPACE (NPSPACE)

Problemas de decisão que podem ser resolvidos em espaço polinomial em uma máquina de Turing determinística (não-determinística).

ALÉM DE NP

$$NL \subseteq P \subseteq NP \subseteq PSPACE = NPSPACE \subseteq EXPTIME \subseteq EXPTIME$$



Demonstrado

$P \subset EXPTIME$

$NL \subset PSPACE$

$PSPACE \subset EXPSPACE$

NP-COMPLETO FRACO

Um problema NP-completo que admite algoritmo pseudopolinomial é dito NP-completo Fraco.



Ex: Problema da Mochila

Itens com peso w_i e valor associado v_i

Mochila com capacidade W.

Existe um carregamento da mochila com itens cuja soma dos valores é, pelo menos, K?

NP-COMPLETO FORTE

Dado um problema NP-completo Π, se a existência de algoritmo pseudopolinomial para Π, acarreta em P = NP, então Π é NP-completo Forte.

Ex: Bin-packing

Itens com volumes w_i

Containers com capacidade c_j

É possível acomodar todos os itens usando *K* containers?

